

Project Proposal Form

Institute of Information Technology (IIT)

University of Dhaka

Student's Name:	Farhan Tausif		
Student's Roll:	1440	Phone:	01305-584050
DeHalu: Agentic Hallucination Detection and Mitigation for local CodeLLMs			
Motivation			
Nowadays, students use local code-generating LLMs (CodeLLMs) because State-Of-The-Art (SOTA) models are expensive and unaffordable for a student-budget, whereas local alternatives provide completely free, unlimited assistance without any subscription fees. However, these models hallucinate and produce incorrect or misleading code. Since the CodeLLMs sound very confident, users may trust and integrate the hallucinated code that fails to compile, misuses APIs, breaks dependencies, or does not meet the actual requirements. In some cases, this can even introduce critical vulnerabilities into a project..			
Project Description			
This project aims to build an agentic tool that acts as a verification gateway for local CodeLLMs, focused on detecting and mitigating hallucinations in generated code before the output is trusted by the user, is called DeHalu. It accepts a user's natural-language programming prompt, generates code with a local CodeLLM, and then passes the generated output through a structured hallucination-detection and mitigation pipeline.			
DeHalu will not rely only on model confidence. Instead of returning raw model output directly to the user, DeHalu provides a verified or repaired result with evidence explaining what was checked and why the final decision was made.			
Proposed Solution			
To achieve reliable code generation, DeHalu implements a multi-stage, agentic verification pipeline by combining static analysis, dynamic sandbox execution, and consensus-based LLM evaluation, the system iteratively detects and mitigates hallucinations.			

The core workflow of the proposed solution is as follows:

- **Prompt Intake and Clarification:** The user provides only a natural-language prompt. The tool clarifies or infers language, framework, runtime, and constraints.
- **Chain-of-Thought [1] -Style Structured Reasoning:** The tool uses structured reasoning prompts to decompose the task, plan verification steps, and guide repair.
- **Code Generation:** A local CodeLLM generates the initial implementation. The tool records the generated code.
- **Claim Extractions:** The generated code is decomposed into checkable claims, that may include imported packages, referenced APIs, etc.
- **Static Analysis:** The code is inspected without execution to identify structural and syntactic issues
- **Sandbox Execution:** The code is then compiled or executed in a controlled environment to detect probable hallucinations such as runtime errors, import failures, timeout risks, and unsafe behaviour, etc. [2].
- **Metamorphic Relation [3] Checks:** DeHalu can generate requirement-preserving input transformations and verify whether the generated code maintains expected behavioral relations.
- **Tool-Backed Validation:** DeHalu Validates that referenced packages and APIs actually exist and are used correctly.
- **LLM-as-a-Judge [4] Verification:** Utilizes a panel consensus of independent verifiers to evaluate if code satisfies the initial requirements, and whether the evidence suggests hallucination.
- **Chain-of-Verification [5] (CoVe):** DeHalu converts generated claims into targeted verification questions and checks them individually.
- **Policy Decision:** All verification signals feed into a policy decision layer that makes a deterministic judgment: accept the code, flag it for clarification, or trigger repair. Deterministic failures may include syntax errors, sandbox failures, unsupported dependencies, etc., which will trigger mitigation.
- **Mitigation and Repair:** The tool focuses on parameter-free mitigation methods [6]. If mitigation is triggered, a repair agent receives the failed code and verification evidence, fixes the issues, and sends the repaired code back through the verification pipeline.

References:

- [1] Y. Lee et al., "Hallucination by Code Generation LLMs: Taxonomy, Benchmarks, Mitigation, and Challenges," arXiv preprint arXiv:2504.20799, 2025.
- [2] J. Cheng et al., "Chain-of-Thought Prompting Obscures Hallucination Cues in Large Language Models: An Empirical Evaluation," arXiv preprint arXiv:2506.17088, 2025
- [3] B. Yang et al., "Hallucination Detection in Large Language Models with Metamorphic Relations," Proceedings of the ACM on Software Engineering (PACMSE), Vol. 2, No. FSE, Article FSE020, 2025
- [4] J. He et al., "LLM-as-a-Judge for Software Engineering: Literature Review, Vision, and the Road Ahead," ACM Transactions on Software Engineering and Methodology, vol. 34, no. 5, 2026.
- [5] S. Dhuliawala et al., "Chain-of-Verification Reduces Hallucination in Large Language Models," Findings of the Association for Computational Linguistics: ACL 2024, pp. 3563–3578, 2024.
- [6] C. Gao et al., "A Systematic Literature Review of Code Hallucinations in LLMs: Characterization, Mitigation Methods, Challenges, and Future Directions for Reliable AI," arXiv preprint arXiv:2511.00776, 2025.

Supervisor's Name: Dr. Kazi Muheymin-Us-Sakib

Signature of the supervisor: _____

Date:

Proposal Presentation Feedback:

- **Static analysis only**
- **Which error raises by hallucination**
- **Why typical solutions are not working on hallucination error**

Saeed Siddik
Assistant Professor,
IIT, DU
19/05/26

Midterm Presentation Feedback: